

BC7215 Driver Library

Application Examples

Index

- Overview.....3
- Learning Remote Control.....3
 - Example Program Function Description.....5
 - Usage.....5
- 2-Channel IR Remote-Controlled Power Outlet.....7
 - Example Program Function Description.....8
 - Usage.....8
- 3. IR Data Transmission.....9
 - Example Program Function Description.....10
 - Usage.....10

Overview

The BC7215 C language driver library examples provide sample applications for the following three uses:

- **Learning remote control**

A learning remote control with four keys. Each key can learn a remote-control command from a different brand.

- **2-channel IR remote-controlled power outlet**

This example can be paired with any remote control and uses unused keys on a household remote control to switch power outputs on or off.

- **IR data transmission**

This example uses the BC7215 as an infrared transceiver for data transmission.

Only minimum resources are needed, they can run on 8-bit micro-controllers with flash as low as 1K.

These examples work best with BC7215, if you are using a BC7215A, which has a 36ms gap time more suitable for ACs, you may need to pay extra attention to keep the key-press short, many A/V device remotes will send repeated frames with an interval less than 36ms, if this happens, the repeats may be treated as part of a long data message by the BC7215A.

Learning Remote Control

This application example uses an MCU together with the BC7215 to build a simple universal learning infrared remote control with four keys. The four keys can each learn a remote-control key in any format. Each key may learn a remote control from a different brand or model and may control a different device.

The example provides designs for two MCUs: STM32F103 and STC8G1K08.

For the STM32F103 MCU, the on-chip USART1 (PA9 and PA10) is connected to the BC7215, and two GPIO pins, PB14 and PB15, are connected to the BUSY and MOD control pins of the BC7215. In addition, four push-button switches are connected between pins PA0-PA3 and ground as key inputs. An LED is connected to pin PC13 to display the operating status. The system block diagram is shown below.

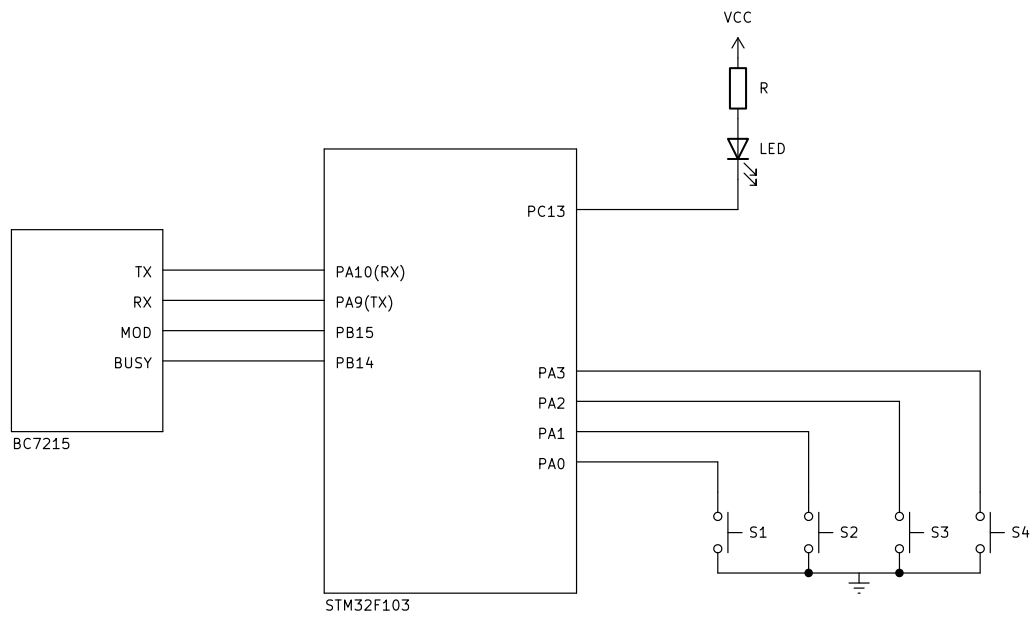


Figure: STM32F103 hardware connection for the learning remote control example.

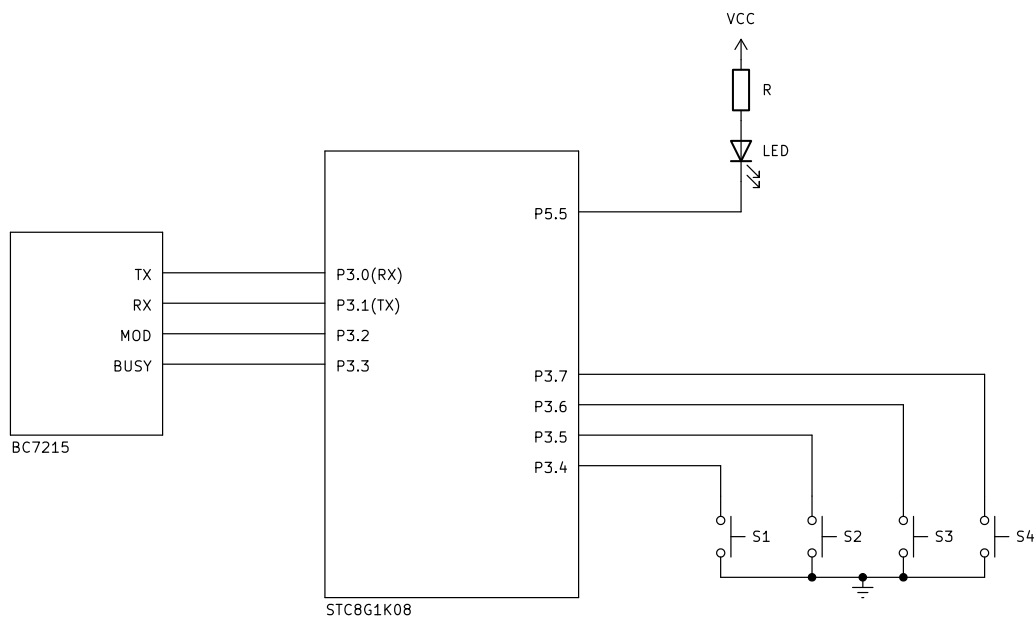


Figure: STC8G1K08 hardware connection for the learning remote control example.

Example Program Function Description

The example program implements a 4-key learning remote control. The program has two states: transmit mode and learning mode. After power-on, it defaults to transmit mode. The LED briefly flashes about once every 1.6 seconds to indicate that it is operating in transmit mode. If a key has already been learned (after power-on the stored content is empty, so the keys do not respond), pressing that key transmits the learned remote-control key function.

In transmit mode, pressing S1 and S2 at the same time enters learning mode. In this state, the program controls the BC7215(A) to enter receive mode, and the LED on PC13 flashes rapidly to indicate that learning mode is active. The user then presses one of S1-S4 to select the key into which the learned content will be stored. After a key is selected, the LED remains steadily on. Once in this state, point the remote control at the infrared receiver of the BC7215 and press a key. If the BC7215(A) successfully receives and decodes the signal, the format of that infrared remote control and the content of the pressed key are saved, and the demo program returns to transmit mode. Pressing the same key again will then copy and send the learned key command.

The demo program is designed so that each key has its own storage for the infrared remote-control format and the raw data. Therefore, it can store four remote-control signals with different formats (from different manufacturers or models). If only different keys of a remote control using the same format are stored, the user may modify the program so that only one copy of the format information is saved, which can reduce memory usage. In addition, for simplicity, the example stores raw data using four maximum-size raw-data-packet arrays. If the user wants to save memory, memory may be dynamically allocated according to the raw data length of each key.

Note: For some remote-control formats, such as RC5 and RC6, the data transmitted each time a key is pressed contains a toggle bit, and this bit changes state on every key press. To keep the learning-remote-control program simple, this function is not implemented. In tests with RC5/RC6 remote controls, if the receiving device remembers the previously received data and expects the polarity of the toggle bit to change, the receiving device may reject the copied original data transmitted by this program. This depends on the design of the receiving device. If the user needs to fully reproduce the function of the original remote control, a toggle-bit inversion function must be added.

This example is only intended to demonstrate BC7215 functions, so it does not implement non-volatile data retention or low-power functions.

Usage

The driver settings for this example are as follows:

```
#define TX_HW_FLOW_CONTROL 0
#define USE_UART_INTERRUPT 1
#define PROCESS_IN_DIFFERENT_THREAD 0
#define ENABLE_RECEIVING 1
#define ENABLE_TRANSMITTING 1
#define ENABLE_FORMAT 1
```

```
#define BC7215_MAX_RX_DATA_SIZE 32
```

This example provides three program examples with exactly the same function. Two are for the STM32F103C8 MCU, and the other is for the STC8G1K08.

The `stm32` directory provides an example based on STMicroelectronics' HAL hardware library. It uses HAL library functions for configuration and hardware operations. After importing the packaged project into STM32 CubeIDE, it can be used directly.

The `stm32\_reg` directory provides a version that does not use a driver library and instead directly operates the registers. In the register-based version, the source program only requires a single file, `main.c`. The only auxiliary files required are the BC7215 driver library itself and STMicroelectronics' register-definition header file, `stm32f10x.h`. It can be used in any STM32 development environment.

The `C51` directory contains the program for the STC8G1K08, including a `main.c` file and a header file that defines special function registers. This version is compiled using the SDCC compiler. Because of compiler limitations, one necessary modification must be made to `bc7215\_lib.c` before compiling: the `\_\_idata` keyword should be added before the statement that defines the UART receive buffer array, so that the upper 128 bytes of internal RAM can be used.

2-Channel IR Remote-Controlled Power Outlet

This application demonstrates the use of the BC7215(A) to build a general-purpose infrared control switch. Every home has various infrared remote controls, and each remote control usually has some keys that are never used. This BC7215 application can use those unused infrared remote-control buttons as infrared remote-control switches. This remote-controlled outlet can use any unused key on any remote control as its control switch.

This example provides application examples for two chips, STM32F103 and STC8G1K08. The circuit schematic diagrams are shown below.

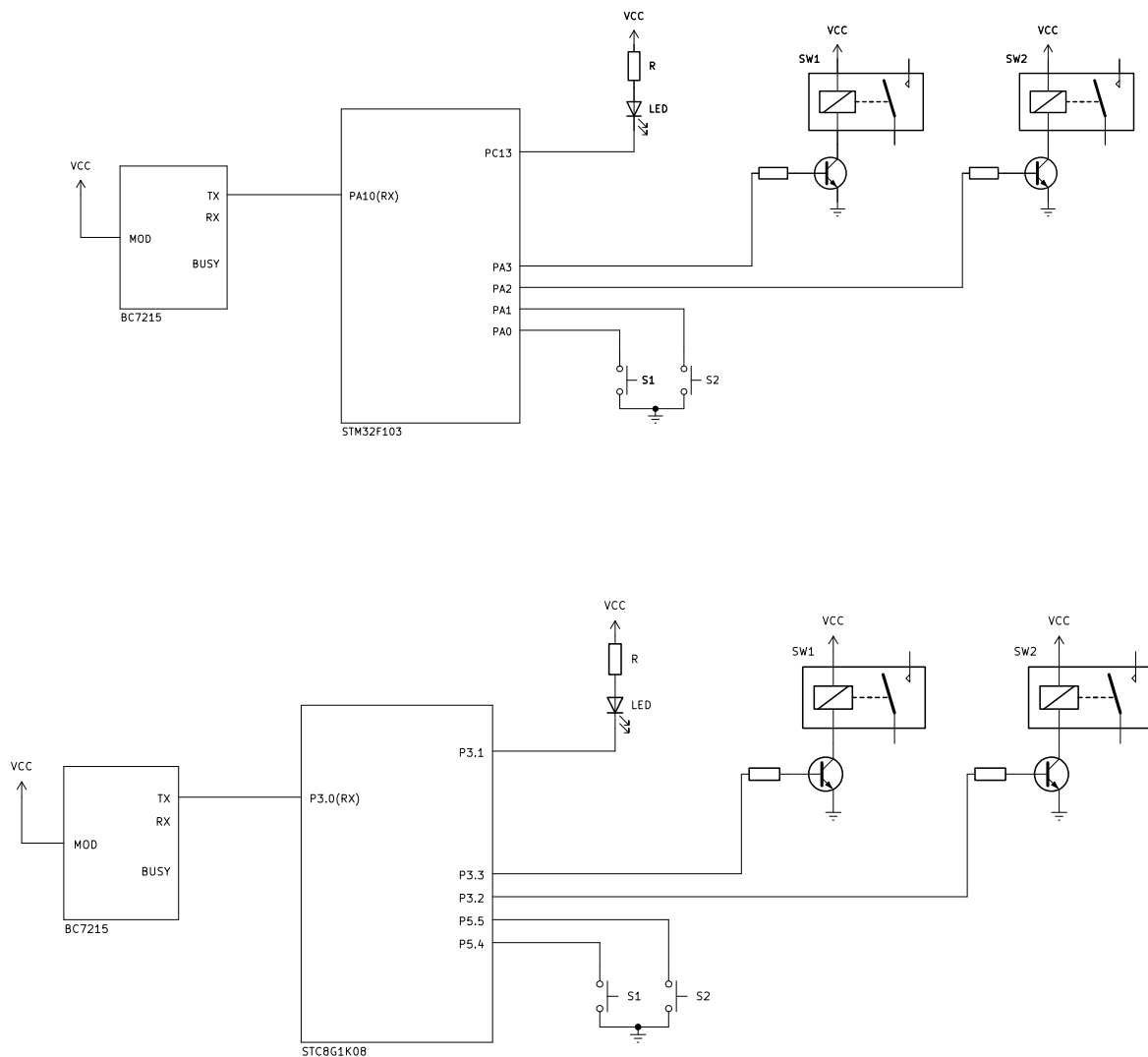


Figure: STM32F103 and STC8G1K08 hardware connections for the 2-channel IR remote-controlled power outlet example.

This example only uses the receive mode of the BC7215. All transmit-related circuits and functions can therefore be omitted, making both the circuit and the program simpler.

Example Program Function Description

The example program implements a 2-channel general-purpose infrared remote-control switch. The circuit itself has two keys, which respectively control the learning functions of the two infrared switch channels. Each channel can learn and remember one key from any remote control. After learning, when the infrared signal of that key is received again, the corresponding relay switch toggles, thereby turning an electrical device on or off.

The program has two states: normal mode and learning mode. After power-on, it defaults to normal mode. The LED briefly flashes about once every 3.2 seconds to indicate that it is operating in normal mode. When the user presses one of the keys, the LED remains steadily on, indicating that the program has entered the learning (pairing) state. At this time, sending an infrared signal from a remote control to the unit stores the signal information, and the program returns to normal mode. Thereafter, if the infrared signal from the same key is received again, the corresponding relay toggles.

This example is only intended to demonstrate BC7215 functions, so it does not implement non-volatile data retention or low-power functions.

Usage

The driver library settings for this application are as follows:

```
#define TX_HW_FLOW_CONTROL 0
#define USE_UART_INTERRUPT 1
#define PROCESS_IN_DIFFERENT_THREAD 0
#define ENABLE_RECEIVING 1
#define ENABLE_TRANSMITTING 0
#define ENABLE_FORMAT 0
#define BC7215_MAX_RX_DATA_SIZE 16
```

This example provides three program examples with exactly the same function. Two are for the STM32F103C8 MCU, and the other is for the STC8G1K08.

The `stm32` directory provides an example based on STMicroelectronics' HAL hardware library. It uses HAL library functions for configuration and hardware operations. After importing the packaged project into STM32 CubeIDE, it can be used directly.

The `stm32\_reg` directory provides a version that does not use a driver library and instead directly operates the registers. In the register-based version, the source program only requires a single file, `main.c`. The only auxiliary files required are the BC7215 driver library itself and STMicroelectronics' register-definition header file, `stm32f10x.h`. It can be used in any STM32 development environment.

The `C51` directory contains the program for the STC8G1K08, including a `main.c` file and a header file that defines special function registers. This version is compiled using the SDCC compiler. Small modifications may be required when using other compilers.

3. IR Data Transmission

This application demonstrates how to use the BC7215 as an infrared transceiver for infrared data transmission. Infrared remote controls are widely used electronic devices, and their communication reliability has been verified. However, their communication speed is relatively low, so this method is suitable for transmitting small amounts of data when speed is not critical, such as sensor data or command data. The example demonstrates both CRC-checked and non-CRC-checked transmission methods.

This example provides application examples for two chips, STM32F103 and STC8G1K08. The circuit schematic diagrams are shown below.

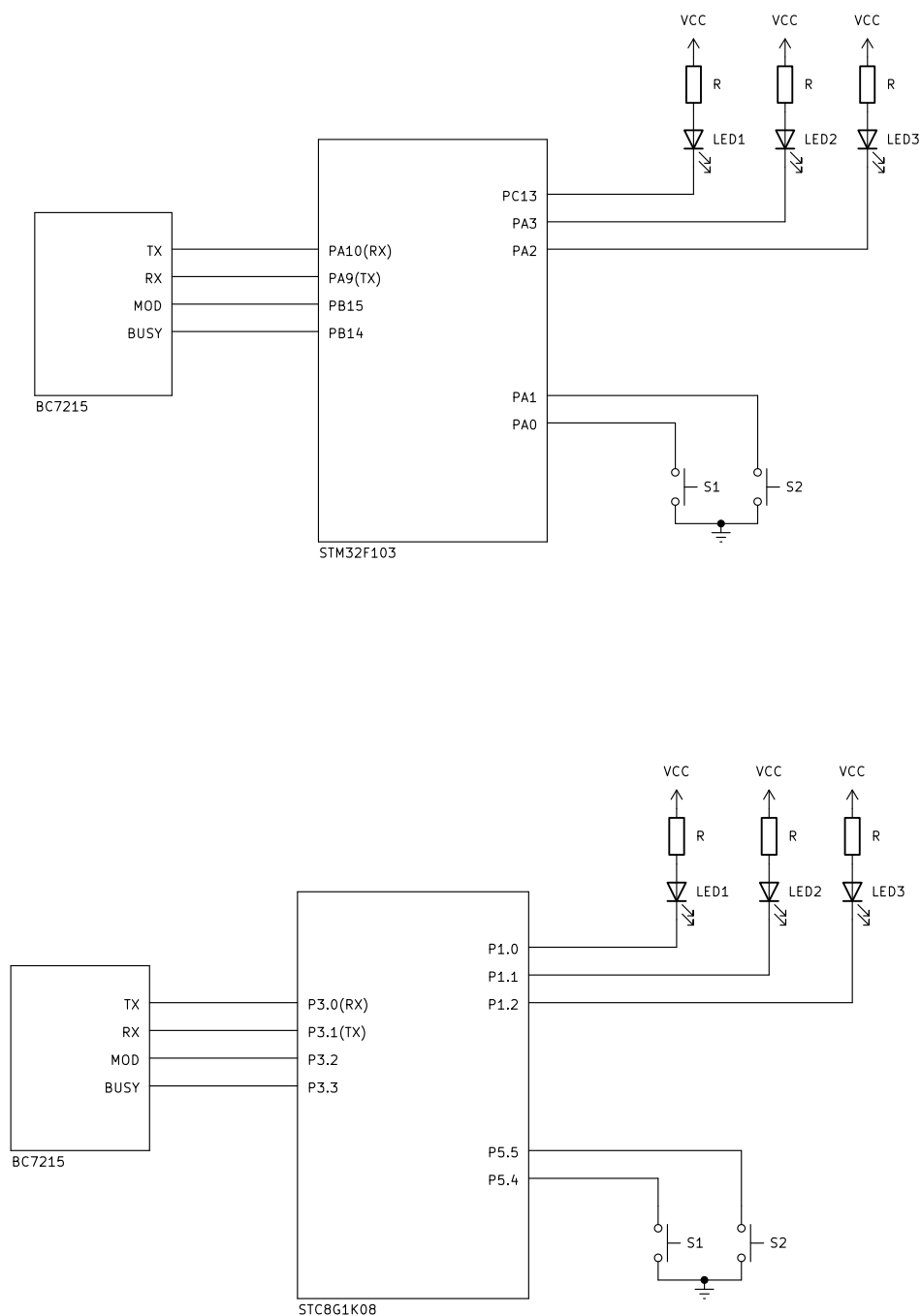


Figure: STM32F103 and STC8G1K08 hardware connections for the IR data transmission example.

This example uses NEC encoding as the infrared transmission encoding method.

Example Program Function Description

This example is only intended to demonstrate the data transmission function, so it has no practical application by itself. The system consists of two keys and three LEDs:

- LED1 - operating-status indicator
- LED2 - indicator for received data with a correct CRC
- LED3 - indicator for received data with an incorrect CRC

After power-on, LED1 flashes once every 3.2 seconds to indicate that the system is operating. By default, the system is in receive mode.

If the S1 key is pressed, the system sends 21 bytes of data by infrared (in this example, the fixed data is the consecutive numbers 0-20). After the numerical data is transmitted, a section of ASCII-code data is transmitted with the content "Hello World". A CRC-8 check value is appended to the end of both data segments. During infrared transmission, LED1 turns on. If the Windows BC7215 demo software is used, and the communication demo function is selected together with the BC7215 demo board, the output data can be seen on the PC. When the software is set to TEXT mode, the 21-byte data above is not displayed because it consists of non-displayable characters, while the text portion can be seen as restored text. When the software is set to HEX mode, two data messages can be seen, 22 bytes and 12 bytes respectively, with the final byte being the CRC byte.

If the S2 key is pressed, the system sends the same data, but no CRC-8 check value is appended at the end. During infrared transmission, LED1 turns on. If the Windows BC7215 demo software is used, and the communication demo function is selected together with the BC7215 demo board, the output data can be seen on the PC. When the software is set to TEXT mode, the 21-byte data above is not displayed because it consists of non-displayable characters, while the text portion can be seen as restored text. When the software is set to HEX mode, two data messages can be seen, 22 bytes and 12 bytes respectively, with the final byte being the CRC byte.

If CRC checking is enabled in the PC software, the user can see that the received data has the correct CRC when S1 is pressed, while the data sent by S2 has an incorrect CRC.

When no key is pressed, the system operates in receive mode. At this time, if an infrared signal is received, regardless of its content, the program performs a CRC check on it. If the check is correct, LED2 flashes once; if the check is incorrect, LED3 flashes once. The user can test this with any ordinary remote control. Because ordinary remote controls do not carry a CRC check value, LED3 will flash. If the BC7215 demo software on a PC is used, or if another identical circuit is used and its S1 key is pressed, LED2 can be seen flashing.

The source program marks the location where data reception is completed. The user may also process the received data in any way required.

Usage

The driver library settings for this application are as follows:

```
#define TX_HW_FLOW_CONTROL 0
#define USE_UART_INTERRUPT 1
#define PROCESS_IN_DIFFERENT_THREAD 0
#define ENABLE_RECEIVING 1
#define ENABLE_TRANSMITTING 1
#define ENABLE_FORMAT 0
#define BC7215_MAX_RX_DATA_SIZE 32
```

This example provides three program examples with exactly the same function. Two are for the STM32F103C8 MCU, and the other is for the STC8G1K08.

The `stm32` directory provides an example based on STMicroelectronics' HAL hardware library. It uses HAL library functions for configuration and hardware operations. After importing the packaged project into STM32 CubeIDE, it can be used directly.

The `stm32\_reg` directory provides a version that does not use a driver library and instead directly operates the registers. In the register-based version, the source program only requires a single file, `main.c`. The only auxiliary files required are the BC7215 driver library itself and STMicroelectronics' register-definition header file, `stm32f10x.h`. It can be used in any STM32 development environment.

The `C51` directory contains the program for the STC8G1K08, including a `main.c` file and a header file that defines special function registers. This version is compiled using the SDCC compiler. Because of compiler limitations, one necessary modification has been made to `bc7215\_lib.c` in the BC7215 driver library: the `\_\_idata` keyword is added before the statement that defines the UART receive buffer array, so that the upper 128 bytes of internal RAM can be used.